

Clasificación de Comandos de Voz: Arquitecturas CNN, CRNN y Transformer con Evaluación de Robustez

Proyecto Final – Curso de Deep Learning
Facultad de Ingeniería
Universidad de Antioquia
Correo: michael.ruiz1@udea.edu.co

Resumen—Este informe presenta un estudio comparativo de tres arquitecturas de deep learning para la clasificación de comandos de voz. Usando el dataset Google Speech Commands (35 clases, 95,246 muestras de entrenamiento) provisto por la competencia de Kaggle *voice-commands-classification-2026*, se diseñaron, entrenaron y evaluaron una CNN Baseline, una CRNN con BiLSTM y un Spectrogram Transformer con Rotary Positional Embeddings (RoPE). Todos los modelos se entrenaron desde cero en Google Colab con GPU NVIDIA T4. La CRNN alcanzó la mejor precisión en validación limpia (94.64 %), seguida del Transformer (94.38 %) y la CNN Baseline (88.94 %). Se realizó una optimización sistemática de hiperparámetros mediante LR Range Test y grid search. El conjunto de test adversarial no pudo evaluarse completamente por falta de etiquetas en su metadata.

Index Terms—Reconocimiento de voz, deep learning, CNN, CRNN, Transformer, RoPE, robustez adversarial, competencia Kaggle

I. INTRODUCCIÓN

I-A. Planteamiento del problema

La tarea consiste en clasificar clips de audio cortos con comandos hablados en inglés dentro de 35 clases. El audio se proporciona como arreglos de forma de onda en formato .npy (mono, 16 kHz, duración variable hasta 1 s). El desafío tiene dos aspectos clave:

1. **Desbalanceo de clases y similitud acústica:** Las 35 clases tienen soporte desigual (1,390 a 3,686 muestras), con varios pares fonéticamente similares (*forward/follow*, *tree/three*, *left/learn*).
2. **Conjunto adversarial:** La competencia provee 10,583 muestras de test adversarial para evaluar robustez ante perturbaciones en la señal. El metadata de este conjunto no incluye etiquetas, simulando una evaluación ciega realista.

I-B. Solución propuesta

Se propone una evaluación comparativa de tres paradigmas arquitectónicos:

- **CNN Baseline:** Pila convolucional pura que trata el Mel-espectrograma como una imagen. Captura patrones locales tiempo- frecuencia. Sirve como línea de base inferior.
- **CRNN (CNN + BiLSTM):** Añade capas LSTM bidireccionales sobre las características convolucionales para

modelar dependencias temporales en la secuencia del espectrograma.

- **Spectrogram Transformer:** Aplica autoatención sobre los frames del espectrograma con RoPE, eliminando la necesidad de codificación posicional absoluta.

Los tres modelos comparten un pipeline de preprocesamiento común (Mel-espectrograma, normalización z-score) implementado en módulos Python compartidos (`models.py`, `data.py`) para garantizar una comparación justa.

I-C. Contexto de la competencia

El dataset proviene de la competencia Kaggle *voice-commands-classification-2026* [7], que proporciona:

- `train/`: 95,246 archivos .npy con `train_metadata.csv` (nombre de archivo, etiqueta).
- `adv_test/`: 10,583 archivos .npy con `test_metadata.csv` (solo nombre de archivo, sin etiquetas).

II. ESTRUCTURA DE LOS NOTEBOOKS

El proyecto se organiza en siete notebooks numerados ejecutables secuencialmente en Google Colab. La Tabla I describe cada uno.

Cuadro I: Descripción de los notebooks del proyecto.

No.	Nombre	Descripción
01	Exploración de datos	Distribución de clases, estadísticas de señales y espectrogramas.
02	Preprocesado	Mel-espectrogramas (n_mels=64), normalización z-score y DataLoaders.
03	CNN Baseline	3 capas Conv2D(32,64,128) + Batch-Norm + MaxPool + FC. Precisión: 88.94 %.
04	CRNN	CNN + BiLSTM(2 capas, hidden=128) + FC. Precisión: 94.64 %.
05	Transformer	Transformer con RoPE + CLS. 4 bloques, dim=120, heads=6. Precisión: 94.38 %.
06	Evaluación de robustez	Carga los 3 modelos y evalúa en validación y test adversarial.
07	Optimización	LR Range Test + grid search (20 % datos, 3 épocas) para hiperparámetros.

Además de los notebooks, el proyecto incluye dos archivos Python compartidos: `models.py` con las definiciones unificadas de las tres arquitecturas y `data.py` con la carga de datos y el split entrenamiento-validación.

III. DATOS Y PREPROCESAMIENTO

III-A. Dataset

El dataset Google Speech Commands v0.02 [1] contiene grabaciones de 35 palabras habladas de aproximadamente un segundo. Las clases incluyen números (*zero-nine*), direcciones (*up, down, left, right, forward, backward*), comandos de control (*stop, go, yes, no, on, off*) y palabras misceláneas (*bird, cat, dog, house, tree, bed, happy, learn, marvin, sheila, visual, wow, follow*).

Cuadro II: Estadísticas del dataset.

Propiedad	Valor
Muestras de entrenamiento	95,246
Muestras de test adversarial	10,583
Número de clases	35
Tasa de muestreo	16,000 Hz
Rango de duración	0.21–1.00 s
Duración media	0.98 s
Clase con más muestras (zero)	3,686
Clase con menos muestras (forward)	1,390

III-B. Disponibilidad de los datos

Los datos se obtienen a través de la competencia de Kaggle <https://www.kaggle.com/competitions/voice-commands-classification-2026>. Para descargarlos es necesario:

1. Tener una cuenta en Kaggle y aceptar las reglas de la competencia.
2. Usar la API de Kaggle: `kaggle competitions download voice-commands-classification-2026`.
3. Descomprimir el archivo descargado, que contiene las carpetas `train/` y `adv_test/` con los archivos `.npz` y los archivos de metadata `train_metadata.csv` y `test_metadata.csv`.

Los notebooks están configurados para descargar los datos automáticamente mediante `kagglehub` si se ejecutan en un entorno sin descarga previa. La estructura esperada de directorios es:

```
dataset/
  train/          # 95,246 archivos .npz
  train_metadata.csv
  adv_test/       # 10,583 archivos .npz
  test_metadata.csv
```

III-C. Pipeline de preprocesamiento

Cada forma de onda cruda pasa por las siguientes transformaciones (implementadas en la clase `AudioToMelSpectrogram` de `models.py`):

1. **Normalización de longitud:** Rellenar o truncar a exactamente 16,000 muestras para consistencia en los batches.
2. **Mel-espectrograma:** Calcular con `n_fft=1024`, `hop_length=256`, `n_mels=64`. Se obtiene un tensor (64, T) con $T \approx 63$ frames.
3. **Compresión logarítmica:** Convertir a escala de decibels mediante `AmplitudeToDB`.
4. **Normalización z-score:** Normalizar cada espectrograma independientemente a media cero y varianza unitaria.

III-D. División de datos

El conjunto de entrenamiento se divide 80/20 estratificado por clase (76,196 train / 19,050 validación) usando `train_test_split` de `sklearn` con `stratify=y`. La estratificación asegura que las clases minoritarias estén representadas proporcionalmente en ambos splits.

IV. ARQUITECTURAS

IV-A. CNN Baseline

La CNN Baseline (`models.py:10`) procesa el Mel-espectrograma como imagen 2D. Arquitectura:

- **Bloque 1:** `Conv2d(1→32, 3×3, padding=1)`, `BatchNorm2d`, `ReLU`, `MaxPool2d(2×2)`, `Dropout2d(0.2)`
- **Bloque 2:** `Conv2d(32→64, 3×3, padding=1)`, `BatchNorm2d`, `ReLU`, `MaxPool2d(2×2)`, `Dropout2d(0.2)`
- **Bloque 3:** `Conv2d(64→128, 3×3, padding=1)`, `BatchNorm2d`, `ReLU`, `MaxPool2d(2×2)`, `Dropout2d(0.2)`
- **Cabeza:** `AdaptiveAvgPool2d(1)`, `Flatten`, `FC(128→64)`, `ReLU`, `Dropout(0.3)`, `FC(64→35)`

Total de parámetros: 173,251. Esta arquitectura no tiene mecanismos de razonamiento temporal – cada frame se procesa independientemente después de que la pila convolucional colapsa la dimensión temporal mediante pooling.

IV-B. CRNN (CNN + BiLSTM)

La CRNN (`models.py:63`) extiende la CNN con capas recurrentes:

- La parte convolucional es idéntica a la CNN Baseline (3 bloques `Conv2D`). Después del Bloque 3, la salida tiene forma (B, 128, 8, T/8).
- El tensor se reordena a (B, T/8, 1024), tratando cada paso temporal como un vector de 1024 dimensiones.
- Una BiLSTM de 2 capas con 128 unidades ocultas por dirección procesa la secuencia. Cada capa produce 256 unidades.
- Mean pooling temporal colapsa la dimensión de secuencia.
- `FC(256→35)` produce los logits finales.

Total de parámetros: 1,303,395. La BiLSTM captura contexto pasado y futuro en cada paso temporal, beneficioso para la discriminación fonética.

IV-C. Spectrogram Transformer con RoPE

El Transformer (`models.py:125`) aplica autoatención sobre los frames del espectrograma usando RoPE [3] en lugar de codificación posicional absoluta.

- **Proyección de entrada:** FC(64 bandas mel $\rightarrow$ 120), aplicado por frame. LayerNorm.
- **Token CLS:** Vector aprendido [CLS] antepuesto a la secuencia de T frames. Su estado oculto final sirve como representación de la secuencia para clasificación.
- **Bloques Transformer ($\times 4$):** Cada bloque contiene LayerNorm + autoatención multi-cabeza (6 cabezas, head_dim=20) con RoPE, MLP (120 $\rightarrow$ 480 $\rightarrow$ 120, GE-LU) y conexiones residuales.
- **Cabeza de clasificación:** Extraer salida [CLS], FC(120 $\rightarrow$ 256), ReLU, Dropout(0.3), FC(256 $\rightarrow$ 35).

Total de parámetros: 724,595. La autoatención computa una matriz de atención entre todos los pares de frames, proporcionando campo receptivo global en una sola capa.

IV-D. RoPE en detalle

RoPE codifica posición rotando los vectores query y key en el espacio complejo. Para la posición t y el par de dimensiones $(2k, 2k+1)$:

$$\Theta(t, 2k) = t \cdot \theta_k^{-2k/d}, \quad \theta_k = 10000 \quad (1)$$

La rotación se aplica como:

$$\text{RoPE}(x_t, t) = \begin{pmatrix} \cos \Theta(t, 2k) & -\sin \Theta(t, 2k) \\ \sin \Theta(t, 2k) & \cos \Theta(t, 2k) \end{pmatrix} \begin{pmatrix} x_t^{(2k)} \\ x_t^{(2k+1)} \end{pmatrix} \quad (2)$$

Esto preserva la estructura del producto punto: $\langle \text{RoPE}(q, t), \text{RoPE}(k, s) \rangle$ depende solo de $(t - s)$, el offset relativo.

V. OPTIMIZACIÓN DE HIPERPARÁMETROS

La optimización se realizó en el Notebook 07 sobre un subconjunto estratificado del 20% de los datos para mantener tiempos viables en Colab gratuito (25 min por ejecución).

V-A. LR Range Test

Se usó el LR Range Test [4]: partiendo de 10^{-7} , el learning rate aumenta exponencialmente en cada mini-batch mientras se registra la pérdida. El LR óptimo es el punto de máximo descenso antes de la divergencia. Resultados:

- CNN Baseline: $2,75 \times 10^{-3}$
- CRNN: $3,98 \times 10^{-3}$
- Transformer: $4,37 \times 10^{-4}$

El Transformer requirió un LR significativamente menor, consistente con la sensibilidad de la autoatención a actualizaciones grandes de pesos.

V-B. Grid Search

Un grid search (3 épocas por combinación) exploró:

- **CNN:** n_mels en {40, 64, 80}, dropout_fc en {0, 0.2, 0.3, 0.5}. Óptimo: n_mels=64, dropout_fc=0.3.
- **CRNN:** lstm_hidden en {128, 256, 384}. Óptimo: 128. Tamaños mayores causaron overfitting.
- **Transformer:** dim en {128, 192, 256}, depth en {3, 4, 6}. Óptimo: dim=256, depth=4. (Los pesos incluidos en el repositorio usan dim=120 por haber sido entrenados antes de la optimización.)

VI. CONFIGURACIÓN DE ENTRENAMIENTO

Todos los modelos se entrenaron en Google Colab con GPU NVIDIA T4 (16 GB VRAM) usando PyTorch 2.11, CUDA 12.8 y torchaudio 2.11.

- Optimizador: AdamW (weight decay 10^{-4})
- Batch size: 64
- Scheduler: CosineAnnealingLR (CRNN, Transformer); ninguno (CNN)
- Early stopping: paciencia 4–5 épocas basado en pérdida de validación
- Épocas máximas: 30
- CNN: Mel-espectrogramas precomputados una vez en float16 y almacenados en CPU (84 s de precómputo + 25 min de entrenamiento)

VII. RESULTADOS

VII-A. Precisión en validación limpia

La Tabla III muestra los resultados principales sobre el conjunto de validación de 19,050 muestras. La CRNN alcanza la mejor precisión, pero el Transformer está a solo 0.26%, sugiriendo que ambos enfoques de modelado temporal superan significativamente a la CNN pura.

Cuadro III: Resultados de validación.

Métrica	CNN	CRNN	Transformer
Precisión (Accuracy)	88.94 %	94.64 %	94.38 %
Macro Precision	0.8922	0.9450	0.9421
Macro Recall	0.8695	0.9395	0.9371
Macro F1-score	0.8778	0.9417	0.9393
Weighted F1-score	0.8886	0.9463	0.9438
Mejor clase (F1)	stop (0.957)	stop (0.977)	six (0.974)
Peor clase (F1)	learn (0.716)	follow (0.892)	learn (0.883)
Parámetros	173,251	1,303,395	724,595

VII-B. Análisis por clase

La CNN baseline muestra una brecha amplia entre mejor y peor clase (0.957 vs. 0.716 F1), mientras que la CRNN y el Transformer comprimen esta brecha a 0.085 y 0.091 respectivamente. Las clases con menos de 300 muestras de validación (*forward*, *learn*, *follow*, *visual*) consistentemente obtienen peores resultados. Los pares más confundidos son acústicamente similares: *forward/follow*, *left/learn* y *tree/three*.

VII-C. Evaluación adversarial

El conjunto de test adversarial (10,583 muestras) no pudo evaluarse porque el archivo `test_metadata.csv` de la competencia no incluye la columna `label` – es intencional, para simular evaluación ciega. Para una evaluación completa de robustez, trabajo futuro debería generar ataques adversariales controlados (FGSM [5], PGD [6]) sobre el conjunto de validación etiquetado y medir la degradación según la magnitud de la perturbación.

VIII. ITERACIONES REALIZADAS

VIII-A. Problemas iniciales de implementación

Se identificaron y corrigieron varios errores críticos durante el proyecto:

1. **Error de dimensión en RoPE (models.py:137):** Los tensores `cos` y `sin` se calculaban usando el número de cabezas de atención (`heads=8`) como longitud de secuencia en lugar de la longitud real `T`. Esto causaba errores silenciosos de dimensión en el embedding rotatorio.
2. **Lazo de entrenamiento del Transformer (nb05):** El modelo recibía audio crudo en lugar de Mel-espectrogramas, causando incompatibilidad de formas en la proyección lineal de entrada.
3. **Rutas de metadata (nb01, nb02):** Las referencias a `metadata.csv` se corrigieron a `train_metadata.csv`.
4. **Alcance de variables (nb02):** `DATA_PATH` estaba definido dentro de un lazo `for`, causando errores de alcance al re-ejecutar.

VIII-B. Refactorización a módulos compartidos

Se refactorizaron las definiciones de modelos en `models.py` y la carga de datos en `data.py`. Esto eliminó la duplicación de código, aseguró un preprocesamiento consistente entre notebooks y permitió que el notebook de evaluación (06) cargara los tres modelos entrenados con una interfaz unificada.

VIII-C. Iteraciones de mejora

1. **Iteración 1 – Notebooks independientes:** Cada modelo se definía dentro de su notebook con código duplicado. Esto generó inconsistencias como el error del Transformer con audio crudo.
2. **Iteración 2 – Refactor:** Unificación en `models.py` y `data.py`. Corrección de errores de RoPE y lazo de entrenamiento.
3. **Iteración 3 – Corrección de EDA:** Eliminación de columna espuria, fijación de frecuencia de muestreo a 16 kHz, corrección de nombres de archivos de metadata.
4. **Iteración 4 – Optimización:** Creación del Notebook 07 con LR Range Test y grid search para justificar empíricamente los hiperparámetros.
5. **Iteración 5 – Entrenamiento final:** Ejecución de notebooks 03–05 en Colab T4. Alineación de hiperparámetros en notebook 06. Versionado de archivos `.pth` en GitHub.

IX. CONCLUSIONES

Se implementaron y compararon tres arquitecturas de deep learning para clasificación de comandos de voz, alcanzando hasta 94.64 % de precisión en validación. Los hallazgos principales son:

- La CRNN con BiLSTM es la arquitectura de mejor rendimiento para esta tarea, combinando extracción de características locales (CNN) con modelado secuencial (BiLSTM).
- El Transformer con RoPE es altamente competitivo, cerrando la brecha con la CRNN a solo 0.26 %. Con una dimensión interna mayor (256 vs. 120), probablemente igualaría o superaría a la CRNN.
- La CNN baseline queda casi 6 puntos atrás, demostrando que el modelado temporal es esencial para esta tarea.
- La optimización sistemática mediante LR Range Test es crítica: los LR óptimos difieren en un orden de magnitud entre arquitecturas.
- El desbalanceo de clases afecta fuertemente las métricas por clase; aumentar datos para clases minoritarias podría mejorar los resultados.
- La falta de etiquetas adversariales resalta la necesidad de evaluación controlada con ataques sintéticos (FGSM, PGD).

REFERENCIAS

- [1] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” *arXiv:1804.03209*, 2018.
- [2] A. Vaswani et al., “Attention Is All You Need,” in *NeurIPS*, 2017.
- [3] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *arXiv:2104.09864*, 2021.
- [4] L. N. Smith, “Cyclical Learning Rates for Training Neural Networks,” in *WACV*, 2017.
- [5] I. J. Goodfellow et al., “Explaining and Harnessing Adversarial Examples,” *arXiv:1412.6572*, 2014.
- [6] A. Madry et al., “Towards Deep Learning Models Resistant to Adversarial Attacks,” *arXiv:1706.06083*, 2017.
- [7] Kaggle, “Voice Commands Classification 2026,” <https://www.kaggle.com/competitions/voice-commands-classification-2026>.
- [8] Repositorio GitHub, https://github.com/maikol0629/dl_voice_command_cl.